More Advanced Query Structures

PAGE NO.....
DATE : /

database: advance_query.

## -- Unions

### 1. UNION vs. UNION ALL:

→ UNION: Combines results and removes duplicates.

→ UNION ALL: Combines results but retains duplicate.

### 2. Use Cases

→ Combining data from different tables with the same structure.

→ Combining results from queries with different filters or conditions.

→ Merging historical data stored in separate tables.

### 3. Requirements

→ The columns in all queries must match in number, order and data type.

→ Column names in the final result set are taken from the first query.

Sales_2022.

| Sale_id | sale_date | product_id | quantity | revenue |
|---|---|---|---|---|
| 1 | 2022-01-15 | 101 | 10 | 100.00 |
| 2 | 2022-02-20 | 102 | 5 | 75.00 |
| 3 | 2022-03-25 | 103 | 8 | 120.00 |
| Int | Date | Int | Int | Decimal (10,2) |

Sales_2023

| Sale_id | sale_date | product_id | quantity | revenue |
|---|---|---|---|---|
| 1 | 2023-01-10 | 101 | 12 | 150.00 |
| 2 | 2023-02-15 | 102 | 6 | 90.00 |
| 3 | 2023-03-20 | 103 | 10 | 200.00 |
| Int | Date | Int | Int | Decimal (10,2) |

PAGE NO.....
DATE / /

## 1. Combine Dates Using UNION.

```
Select
    sale_id,
    sale_date,
    product_id,
    quantity,
    revenue
From
    sales_2022

UNION

Select
    sale_id,
    sale_date,
    product_id,
    quantity,
    revenue
From
    Sales_2023
order by
    sale_date;
```

output

| sale_id | sale_date | product_id | quantity | revenue |
|---|---|---|---|---|
| 1 | 2022-01-15 | 101 | 10 | 1000 |
| 2 | 2022-02-20 | 102 | 5 | 75.00 |
| 3 | 2022-03-23 | 103 | 8 | 120.00 |
| 1 | 2023-01-10 | 101 | 12 | 150.00 |
| 2 | 2023-02-13 | 102 | 6 | 90.00 |
| 3 | 2023-03-20 | 103 | 10 | 200.00 |

2 Advanced Examples with CTE and Union.

```
with sales_combined As (
    Select
        sale_date,
        product_id,
        quantity,
        revenue
        Year (sale_date) As sale_year
    From
        sales_2022

    UNION ALL
    Select
        sale_date,
        product_id,
        quantity,
        revenue,
        Year (sale_date) As sale_year
    From
        sales_2023.
),
top_products As (
    Select
        sale_year,
        product_id,
        SUM (quantity) As total_quantity.
    RANK() over (partition by sale_year Order by
                 SUM (quantity) Desc) As quantity_rank.
    From
        sales_combined
    Group by
        sale_year, product_id.
)
```

)

```
Select
    Sale_year,
    product_id,
    total_quantity
From
    top_products
Where
    quantity_rank = 1
Order by
    sale_year;
```

sales combined

| sale_date | product_id | quantity | revenue | sale_year |
|---|---|---|---|---|
| 2022-01-15 | 101 | 10 | 100.00 | 2022 |
| 2022-02-20 | 102 | 5 | 75.00 | 2022 |
| 2022-03-25 | 103 | 8 | 120.00 | 2022 |
| 2023-01-10 | 101 | 12 | 150.00 | 2023 |
| 2023-02-15 | 102 | 6 | 90.00 | 2023 |
| 2023-03-20 | 103 | 10 | 200.00 | 2023 |

top_product

| sale_year | product | total_quantity | quantity_rank |
|---|---|---|---|
| 2022 | 101 | 10 | 1 |
| 2022 | 103 | 8 | 2 |
| 2022 | 102 | 5 | 3 |
| 2023 | 101 | 12 | 1 |
| 2023 | 103 | 10 | 2 |
| 2023 | 102 | 6 | 3 |

PAGE NO.....
DATE / /

output,

| Sale_year | product_id | total_quantity |
|---|---|---|
| 2022 | 101 | 10 |
| 2023 | 101 | 12 |

PAGE NO.....
DATE / /

-- self-join to Determine To-Date Maximum.

-> A self-join in SQL is a technique where a table is joined to itself. This is useful when you need to compare rows within the same table. For example, you might want to compare each row to all previous rows to determine a "to-date-maximum" value, such as the highest sales to a specific date.

->The highest up to that date is achieved by,

1.) Summarizing sales by date using a CTE (Common Table Expression).

2.) Performing a self-join on the summarized data, comparing each date to all previous dates.

3.) Using aggregation (Max()) and conditional logic (CASE) to identify when a new sales record is set.

table name: ~~employee_salary_summary~~. employee_salaries.

| employee_id | salary_date | salary_amount |
|---|---|---|
| 1 | 2023-01-01 | 5000.00 |
| 1 | 2023-02-01 | 5500.00 |
| 1 | 2023-03-01 | 6000.00 |
| 1 | 2023-04-01 | 5500.00 |
| 1 | 2023-05-01 | 7000.00 |
| 2 | 2023-01-01 | 4500.00 |
| 2 | 2023-02-01 | 5000.00 |
| 2 | 2023-03-01 | 4800.00 |
| 2 | 2023-04-01 | 5200.00 |
| 2 | 2023-05-01 | 5800.00 |

Int Date Decimal(10,2)

* Confusion: need alot of repeatation

PAGE NO.....
DATE / /

```
with employee_salary_summary As (
  Select
      employee_id,
      Salary_date,
      salary_amount.
  From employee_salaries.
  )

Select
    as. employee_id,
    as. salary_date,
    as. salary_amount,
    Max (prev. as. salary_amount) As previous_max_salary,
    Case
1-  [ When as. salary_amount > Max (prev_as. salary_amount)
      Then 'Yes' Else 'NO'
    End as salary_record_set.
From employee_salary_summary As as.
Left Join employee_salary_summary As prev. as.
2-  [ ON as. employee_id = prev_as. employee_id.
    AND prev_as. salary_date < as. salary_date.
Group By
    as. employee_id,
    as. salary_date,
    as. salary_amount.
Order by
    as. employee_id,
    as. salary_date;
```

output

| employee_id | salary_date | salary_amount | previous_max_salary | salary_record_set |
|---|---|---|---|---|
| 1 | 2023-01-01 | 5000.00 | Null | No |
| 1 | 2023-02-01 | 5500.00 | 5000.00 | Yes |
| 1 | 2023-03-01 | 6000.00 | 5500.00 | Yes |
| 1 | 2023-04-01 | 5500.00 | 6000.00 | No |
| 1 | 2023-05-01 | 7000.00 | 6000.00 | Yes |
| 2 | 2023-01-01 | 4800.00 | Null | No |
| 2 | 2023-02-01 | 5000.00 | 4300.00 | Yes |
| 2 | 2023-03-01 | 4800.00 | 5000.00 | No |
| 2 | 2023-04-01 | 5200.00 | 5000.00 | Yes |
| 2 | 2023-05-01 | 5300.00 | 5200.00 | Yes |

-> New time

# Counting New vs. Returning Customers by Week.

PAGE NO.....
DATE / /